Plataformas de Desenvolvimento

24/25

Mestrado em Informática e Sistemas

1

Objectives

- Why Git?
- Git main commands
- Best Practices
- GitHub Actions

2

Assumptions

- Almost all examples will be made through command line
- Consistency between Linux/Windows/MAC

Why Source control?

- Backup
- Versioning / History
- Rollback / Undo Changes
- Comparing stuffs
- Teamwork
- Identify the greatest bug creator ☺
- Isolate new features
- Easier to change the context
- Code review

Who needs Source control?

- Software developers
- Administrators (devops)
- Graphic Artists
- Web designers
- …

- Everyone that needs backup, history and some sort of diff with previous versions

How to achieve Source Control?

- Two main types:
  - Centralized
  - Distributed (DVCS)

Centralized: Subversion (SVN), CVS …

Distributed: Git, Mercurial
- Most operations are local (good if you don't have internet access,vpn,…)
- Central server not mandatory. Only to push changes to a central point.
- Clone of a repository retrieves all historical data

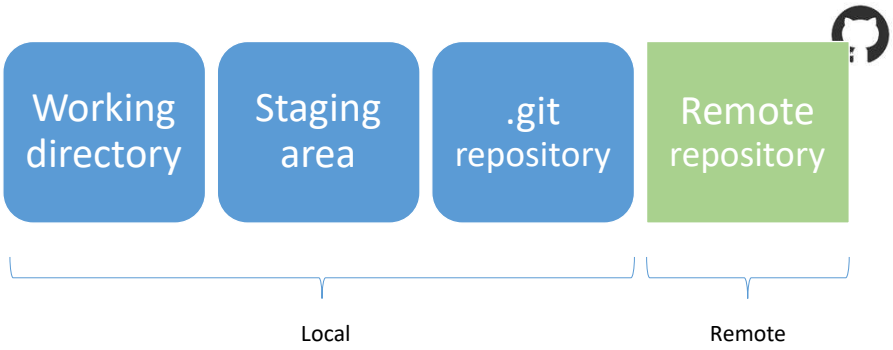
Why GIT?

- Distributed source control system
- Most operations are local
- Scales
- Open source
- Active community
- Started by Linux projects

GIT concepts

- Repository contains files, historical info, configs
- .git/ folder with git info such as versions, etc
- Three states for Git:
  - Working directory
  - Staging area – pre-commit area
  - Commit – Git repository (.git folder)
- Remote repository (github)
  - Usually by default Main branch

GIT concepts



GIT concepts

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

A clean repository

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

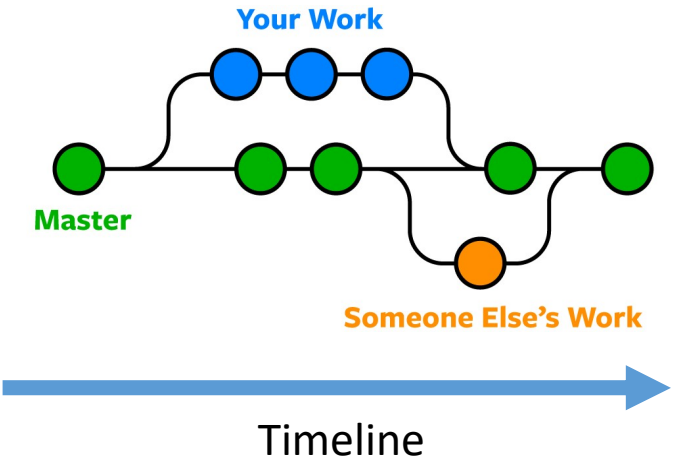
        new file:   new_file.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README.md
```

A repository with changes to be commited and to be staged

GIT concepts



11

GIT basic commands

Starting a new project:

- Empty git repository (git init)
- Existing source (git init)
- Github (fork and clone)

Git basic workflow
- add, commit , push and pull operations

Files can be renamed, moved or deleted

History and aliases

12

GIT basic commands

Starting a new project:

- Empty git repository (git init)

```
PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests
$ git init git_tests
Initialized empty Git repository in D:/git/git_tests/git_tests/.git/
```

```
PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests/git_tests (master)
$ ll -a
total 4
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 ./
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 ../
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 .git/
```

- Existing source (git init)
- Github (fork and clone)

GIT basic commands

Starting a new project:

Empty git repository (git init)

```
PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests
$ git init git_tests
Initialized empty Git repository in D:/git/git_tests/git_tests/.git/
```

```
PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests/git_tests (master)
$ ll -a
total 4
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 ./
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 ../
drwxr-xr-x 1 PTIN+10052766 4266656257 0 Apr 16 22:29 .git/
```

Adding files to repository

```
PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests/git_tests (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        teste_add.txt

nothing added to commit but untracked files present (use "git add" to track)

PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests/git_tests (master)
$ git add teste_add.txt

PTIN+10052766@TD50002405 MINGW64 /d/git/git_tests/git_tests (master)
$ git status
On branch master

No commits yet

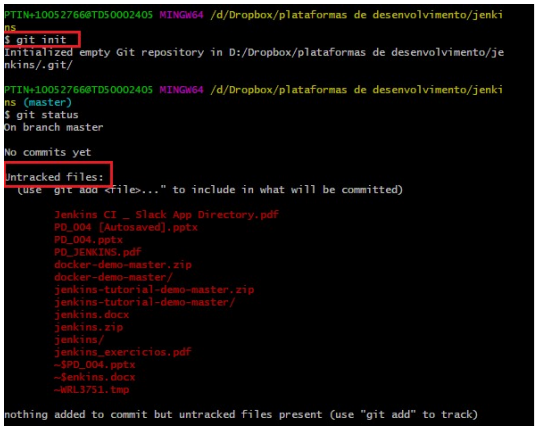
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   teste_add.txt
```

GIT basic commands

Starting a new project:

- Existing source (git init)

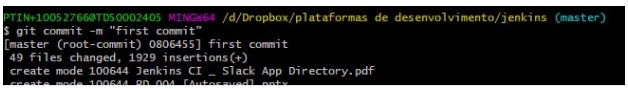


Adding files to repository





Commit:

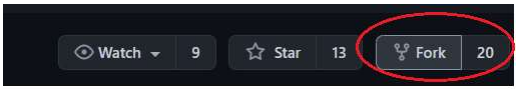


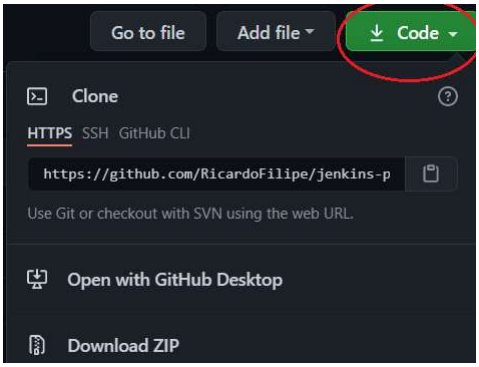
15

GIT basic commands

Starting a new project:

- Github (fork and clone)





16

GIT basic commands

Git basic workflow
- add, commit , push and pull operations

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento
$ git clone https://github.com/RicardoFilipe/jenkins-pd-nodejs.git
Cloning into 'jenkins-pd-nodejs'...
remote: Enumerating objects: 56, done.
remote: Counting objects: 100% (56/56), done.
remote: Compressing objects: 100% (53/53), done.
remote: Total 56 (delta 22), reused 10 (delta 0), pack-reused 0
Unpacking objects: 100% (56/56), done.
```

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main
$ vi README.md

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main
$ git add README.md
```

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (mair
$ git add README.md

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (mair
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        modified:   README.md


PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (mair
$ git commit -m "change readme file"
[main b5dc627] change readme file
 1 file changed, 4 insertions(+)
```

17

GIT basic commands

Git basic workflow
- add, commit , push (or fetch + merge) and pull operations

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git pull origin main
From https://github.com/RicardoFilipe/jenkins-pd-nodejs
 * branch            main       -> FETCH_HEAD
Already up to date.
```

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git push
Username for 'https://github.com': RicardoFilipe
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 468 bytes | 234.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/RicardoFilipe/jenkins-pd-nodejs.git
   8dec206..29e12d9  main -> main
```

Beware of the `git push -force!`
Overwrite the commit history on the remote with your local one.

18

GIT basic commands

Files can be renamed, moved or deleted

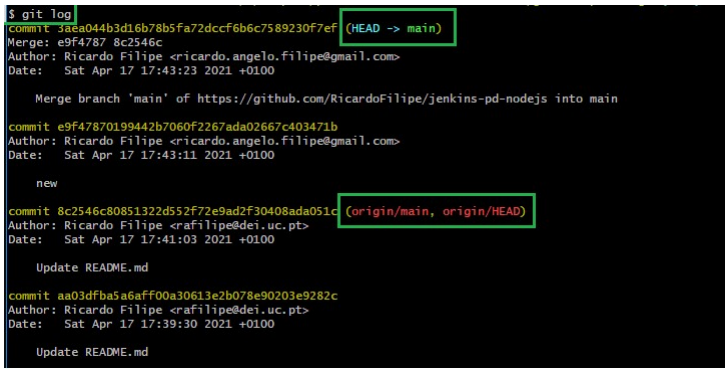
git mv file_name new_file_name

git rm filename

Reset/Undo the HEAD index:
git reset

GIT basic commands

$git log
$git log --abbrev-commit
$git log --oneline --graph --decorate
$git log <commit_id_1>...<commit_id_2>
$git log --since="2 days ago"



GIT basic commands

Git alias to create new commands.

“git hist“ will show a customized git log.

```
PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git config --global alias.hist "log --all --graph --decorate --oneline"

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ cat ~/.gitconfig
[alias]
        hist = log --all --graph --decorate --oneline

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git hist
*   3aea044 (HEAD -> main) Merge branch 'main' of https://github.com/RicardoFilipe/jenkins-pd-nodejs into main
|\
| * 8c2546c (origin/main, origin/HEAD) Update README.md
* | e9f4787 new
|/
* aa03dfb Update README.md
*   29e12d9 changes
|\
| * 8dec206 Update README.md
* | b5dc627 change readme file
```


GIT basic commands

Git remove unwanted files.
Create a new file called .gitignore with unwanted:

- specific files
- folders
- files that match regular expression (e.g. *.log)

```
$ git add .gitignore
warning: LF will be replaced by CRLF in .gitignore.
The file will have its original line endings in your working directory

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git commit -m "git ignore"
[main f2da7b0] git ignore
 1 file changed, 1 insertion(+)
 create mode 100644 .gitignore
```

```
$ cat .gitignore
*.log
```

GIT basic commands

git help <command> is helpfull

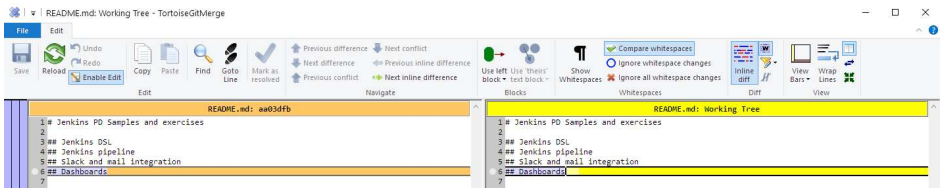
23

GIT diff tools

git diff alows to understand changes between commits.

Easier to use visual diff tools, such as winmerge.

```
$ git diff
diff --git a/README.md b/README.md
index 6021f15..1ee2c94 100644
--- a/README.md
+++ b/README.md
@@ -3,4 +3,4 @@
 ## Jenkins DSL
 ## Jenkins pipeline
 ## Slack and mail integration
-## Dashboards
+## Dashboards
```



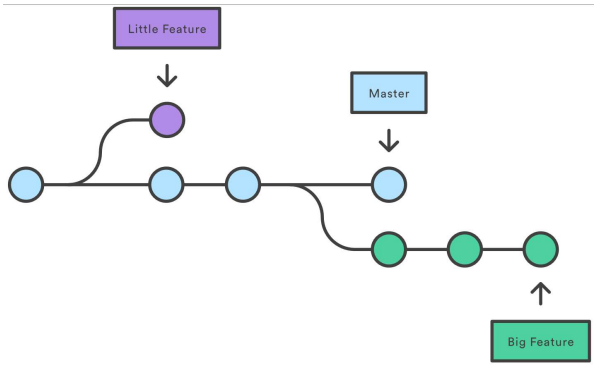
24

GIT branch

Branch is an independent development or feature.

Allows to have a new working/stage/commit area to a specifc task.

git branch command lets you create, list, rename, and delete branches



25

GIT branch

```
$ git branch newbranch

PTIN+10052766@TD50002405 MINGW64 /d/Dropbox/plataformas de desenvolvimento/jenkins-pd-nodejs (main)
$ git checkout newbranch
Switched to branch 'newbranch'
```

```
$ git branch -d newbranch
Deleted branch newbranch (was f2da7b0).
```

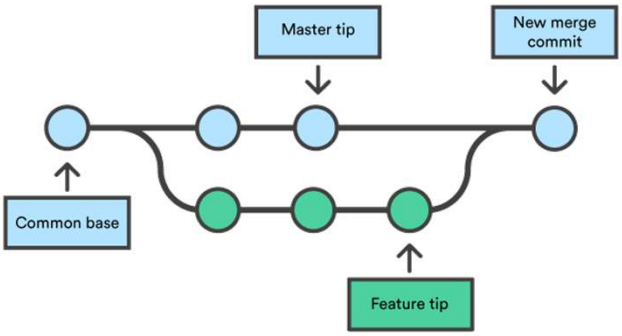
26

GIT merge

Git merge will combine multiple sequences of commits into one unified history.

Normally used to merge 2 branches.

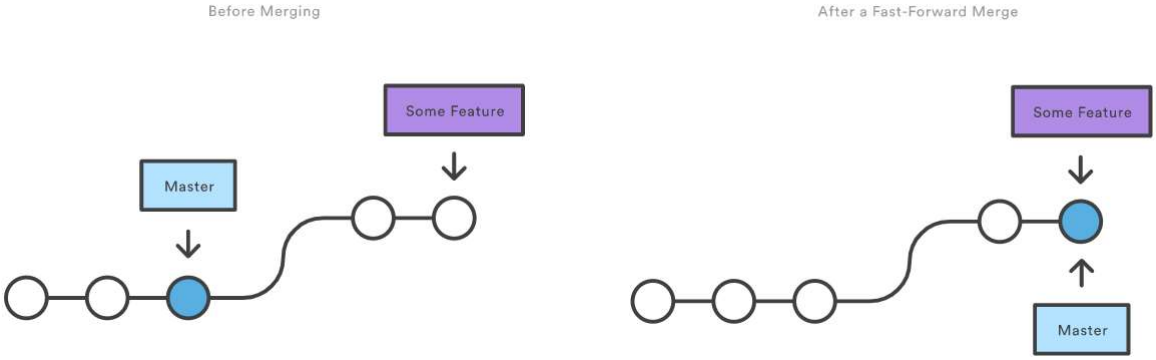
If Git encounters a piece of data that is changed in both histories it will be unable to automatically combine them. This scenario is a version control conflict and Git will need user intervention to continue.



27

GIT merge

Fast forward Merge
A fast-forward merge can occur when there is a linear path from the current branch tip to the target branch (e.g. git pull == git fetch + git merge)



28

GIT merge

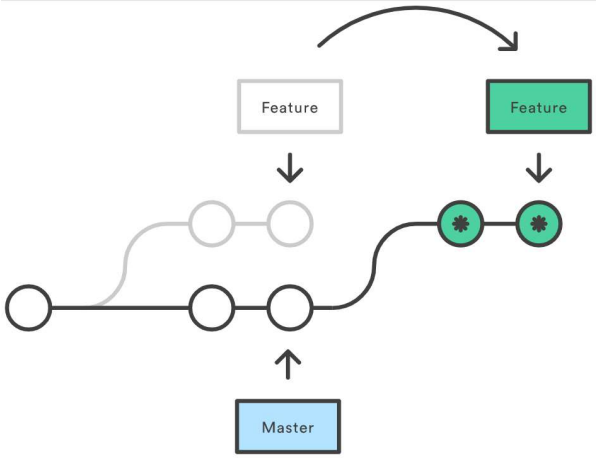
```
# Start a new feature
git checkout -b new-feature master
# Edit some files
git add <file>
git commit -m "Start a feature"
# Edit some more files
git add <file>
git commit -m "Finish a feature"
# Merge in the new-feature branch
git checkout master
git merge new-feature
git branch -d new-feature
```

29

GIT rebase

Rebasing is the process of moving or combining a sequence of commits to a new base commit. Rebasing is most useful and easily visualized in the context of a feature branching workflow.

Allows to have the last commits of
a main branch under our feature branch



30

GIT rebase

Git rebase in standard mode will automatically take the commits in your current working branch and apply them to the head of the passed branch.

```
git rebase <base>
```

Git rebase with the -i flag begins an interactive rebasing session. Instead of blindly moving all of the commits to the new base, interactive rebasing gives you the opportunity to alter individual commits in the process.

```
git rebase --interactive <base>
```


GIT stash

git stash temporarily shelves (or *stashes*) changes you've made to your working copy so you can work on something else, and then come back and re-apply them later on.

Stashing is handy if you need to quickly switch context and work on something else, but you're mid-way through a code change and aren't quite ready to commit.

GIT stash – create stash

```
$ git status
On branch master
Changes to be committed:

    new file:   style.css

Changes not staged for commit:

    modified:   index.html

$ git stash
Saved working directory and index state WIP on master:
5002d47 our new homepage
HEAD is now at 5002d47 our new homepage

$ git status
On branch master
nothing to commit, working tree clean
```


GIT stash – reaply code to branch

```
$ git status
On branch master
nothing to commit, working tree clean
$ git stash pop
On branch master
Changes to be committed:

   new file:   style.css

Changes not staged for commit:

   modified:   index.html

Dropped refs/stash@{0} (32b3aa1d185dfe6d57b3c3cc3b32cbf3e380cc6a)
```

GIT stash – multiple stash

```
$git stash save "simple changes"
#some changes
$git stash save “changes 1"
# more changes
$git stash save “changes 2"
$git stash list
$git stash show stash@{1}
$git stash list
$git stash apply stash@{1}
```


GIT tag

Tags are ref's that point to specific points in Git history.

Tagging is generally used to capture a point in history that is used for a marked version release (i.e. v1.0.1). A tag is like a branch that doesn’t change. Unlike branches, tags, after being created, have no further history of commits.

```
git tag <tagname>
```

Annotated tags are stored as full objects in the Git database. To reiterate, They store extra meta data such as: the tagger name, email, and date. To github you must force tags to be push (e.g. `git push origin master –tags`)

```
git tag -a v1.4
```

GIT

Learning curve



42

Version Control Practices

Version Control contains the source code for the project and mantains current and historical state of the code

Version Control used by all teams of the project:

- Developers
- Operations
- QA
- Security
- …

43

Version Control Practices

1. Everything should be on the version control system
2. Commit Often (e.g. smaller pieces of code)
3. Use easy to understand commit messages (and link with a project management tool e.g. Jira)
4. Don’t commit code that you know it’s not working (it might broke the CI/CD pipeline)

Version Control Practices

5. Define a branching strategy depending of your project:
    - Trunk-based Development
        - Developers work in a single main branch (trunk), integrating small, frequent changes directly into the trunk.
        - Branches last typically only hours
        - What about big system changes?
            - Branch by abstraction
                - a technique for making a large-scale change to a software system in gradual way that allows you to release the system regularly while the change is still in-progress.

Version Control Practices

5. Define a branching strategy depending of your project:
    - Feature Branching (Gitflow)
        - Developers create long-lived feature branches, merging them back to the main branch only after features are complete and reviewed.
        - Separate primary branch lines for development, hotfixes, features, and releases

DevOps Report:
Trunk-based development was highly correlated to high-performing teams

Atlassian:
Gitflow has fallen in popularity in favor of trunk-based workflows, which are now considered best practices for modern continuous software development and DevOps practices. Gitflow also can be challenging to use with CI/CD.

46

GitHub actions - Introductions

- Built-in automation tool in GitHub
- Enables CI/CD, workflow automation, and custom scripts
- YAML-based configuration
- Usefull for CI/CD Pipelines

47

Why Use GitHub Actions

- Integrated with GitHub (No external CI/CD needed)
- Event-driven automation (Runs on push, pull request, etc.)
- Extensive marketplace (Reusable actions available)
- Self-hosted or GitHub-hosted runners

Key Concepts

- Workflows: Automated processes triggered by events
- Jobs: Tasks executed in a workflow
- Steps: Individual commands/scripts inside a job
- Runners: Virtual machines that execute workflows
- Marketplace Actions: Pre-built actions from the community

Basic Workflow

The workflows are located in the git repositor in .github/workflows/<workflow>.yml

```yaml
name: CI Pipeline
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Run Tests
        run: npm test
```


Other Features

Matrix Builds

Secrets & Environment Variablers

Self-Hosted Runners

https://docs.github.com/en/actions

Some bibliography

- https://git-scm.com/

- https://www.perforce.com/blog/vcs/git-vs-svn-what-difference
- https://www.baeldung.com/cs/git-vs-svn
- https://www.tabnine.com/blog/svn-vs-git/
- https://www.openhub.net/repositories/compare
- https://www.atlassian.com/git/tutorials
- 2023_final_report_sodr.pdf
- Branch By Abstraction